# DESIGN AND ANALYSIS OF ALGORITHMS

## DAA LABORATORY PROGRAMS

### Algorithm Analysis Fundamentals

| | |
|---|---|
| **Name** | NOELSHAUN DERIL.W |
| **Register Number** | 192324174 |
| **Course Code** | CSA0617 |
| **Faculty** | Dr S Pradeep Kumar |

# EXPERIMENT 1

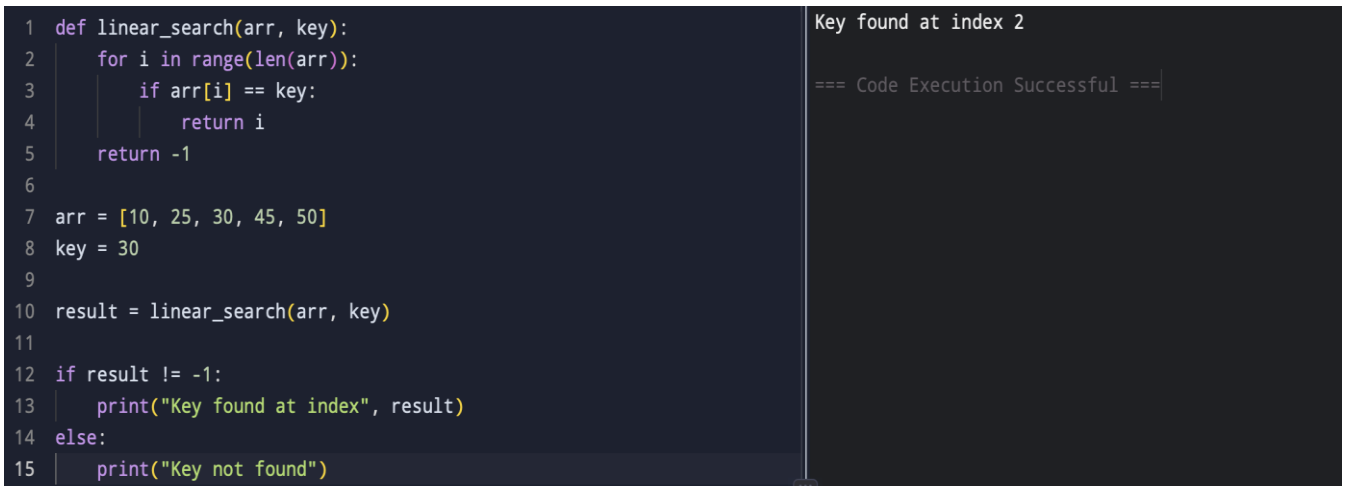
# LINEAR SEARCH (ITERATIVE)

## Python Program

```
def linear_search(arr, key):
    for i in range(len(arr)):
        if arr[i] == key:
            return i
    return -1

arr = [10, 25, 30, 45, 50]
key = 30

result = linear_search(arr, key)

if result != -1:
    print("Key found at index", result)
else:
    print("Key not found")
```

## Execution Screenshot

### EXECUTION SCREENSHOT



# EXPERIMENT 2

# BINARY SEARCH (RECURSIVE)

## Python Program

```
def binary_search(arr, low, high, key):
    if low <= high:
        mid = (low + high) // 2

        if arr[mid] == key:
            return mid
        elif arr[mid] < key:
            return binary_search(arr, mid + 1, high, key)
        else:
            return binary_search(arr, low, mid - 1, key)

    return -1

arr = [5, 10, 15, 20, 25]
key = 20

result = binary_search(arr, 0, len(arr) - 1, key)

if result != -1:
    print("Key found at index", result)
else:
    print("Key not found")
```

## Execution Screenshot

**EXECUTION SCREENSHOT**

```python
def binary_search(arr, low, high, key):
    if low <= high:
        mid = (low + high) // 2

        if arr[mid] == key:
            return mid
        elif arr[mid] < key:
            return binary_search(arr, mid + 1, high, key)
        else:
            return binary_search(arr, low, mid - 1, key)

    return -1
```

```
Key found at index 3

=== Code Execution Successful ===
```

# EXPERIMENT 3

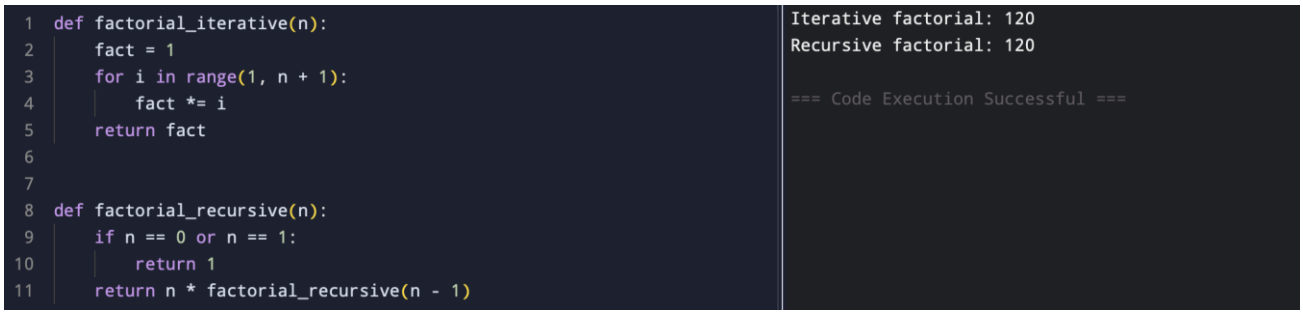
# FACTORIAL (ITERATIVE VS RECURSIVE)

## Python Program

```
def factorial_iterative(n):
    fact = 1
    for i in range(1, n + 1):
        fact *= i
    return fact


def factorial_recursive(n):
    if n == 0 or n == 1:
        return 1
    return n * factorial_recursive(n - 1)


n = 5

print("Iterative factorial:", factorial_iterative(n))
print("Recursive factorial:", factorial_recursive(n))
```

## Execution Screenshot

### EXECUTION SCREENSHOT



# EXPERIMENT 4

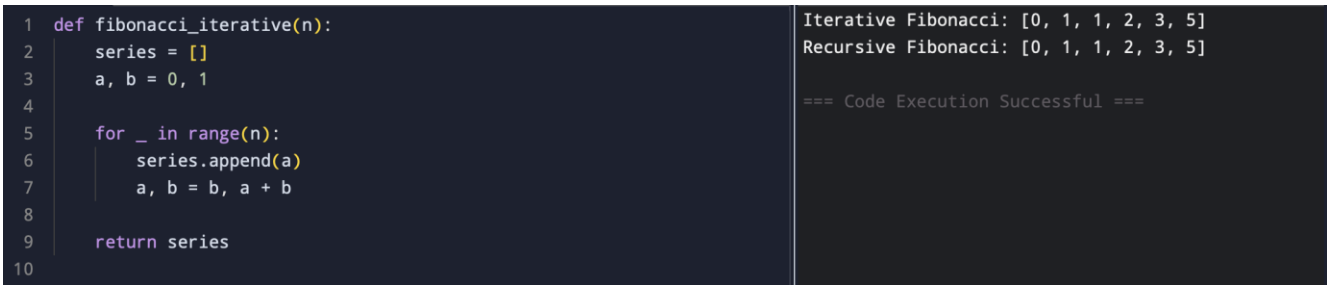
# FIBONACCI SERIES

## Python Program

```
def fibonacci_iterative(n):
    series = []
    a, b = 0, 1

    for _ in range(n):
        series.append(a)
        a, b = b, a + b

    return series


def fibonacci_recursive(n):
    if n <= 1:
        return n
    return fibonacci_recursive(n - 1) + fibonacci_recursive(n - 2)


n = 6

print("Iterative Fibonacci:", fibonacci_iterative(n))
print("Recursive Fibonacci:", [fibonacci_recursive(i) for i in range(n)])
```

## Execution Screenshot

### EXECUTION SCREENSHOT



# EXPERIMENT 5

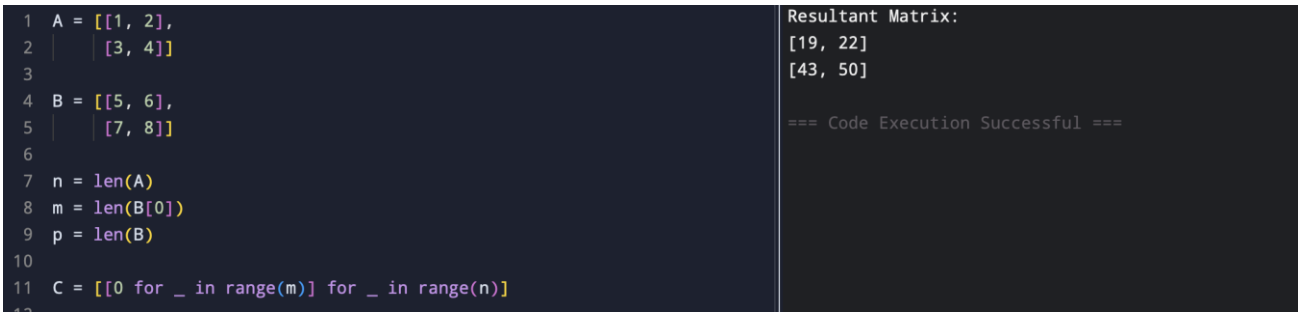
# MATRIX MULTIPLICATION

## Python Program

```
A = [[1, 2],
     [3, 4]]

B = [[5, 6],
     [7, 8]]

n = len(A)
m = len(B[0])
p = len(B)

C = [[0 for _ in range(m)] for _ in range(n)]

for i in range(n):
    for j in range(m):
        for k in range(p):
            C[i][j] += A[i][k] * B[k][j]

print("Resultant Matrix:")
for row in C:
    print(row)
```

## Execution Screenshot

### EXECUTION SCREENSHOT



# EXPERIMENT 6

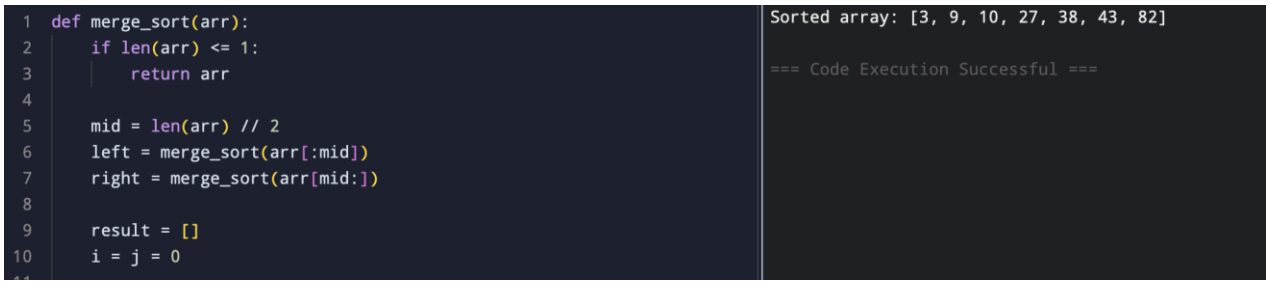
# MERGE SORT

## Python Program

```
def merge_sort(arr):
    if len(arr) <= 1:
        return arr

    mid = len(arr) // 2
    left = merge_sort(arr[:mid])
    right = merge_sort(arr[mid:])

    result = []
    i = j = 0

    while i < len(left) and j < len(right):
        if left[i] <= right[j]:
            result.append(left[i])
            i += 1
        else:
            result.append(right[j])
            j += 1

    result.extend(left[i:])
    result.extend(right[j:])

    return result


arr = [38, 27, 43, 3, 9, 82, 10]

print("Sorted array:", merge_sort(arr))
```

## Execution Screenshot

### EXECUTION SCREENSHOT



# EXPERIMENT 7

# QUICK SORT

## Python Program

```
def quick_sort(arr):
    if len(arr) <= 1:
        return arr

    pivot = arr[-1]
    left = [x for x in arr[:-1] if x <= pivot]
    right = [x for x in arr[:-1] if x > pivot]

    return quick_sort(left) + [pivot] + quick_sort(right)


arr = [10, 7, 8, 9, 1, 5]

print("Sorted array:", quick_sort(arr))
```

## Execution Screenshot

### EXECUTION SCREENSHOT

# EXPERIMENT 8

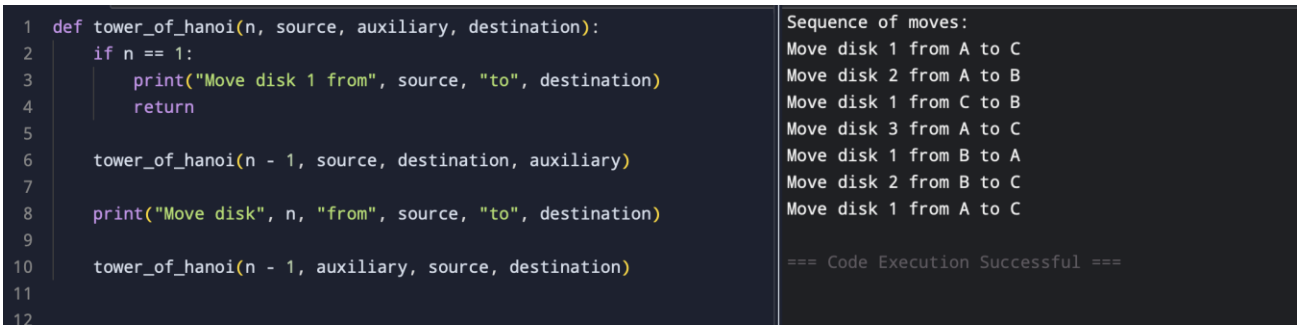
# TOWER OF HANOI

## Python Program

```
def tower_of_hanoi(n, source, auxiliary, destination):
    if n == 1:
        print("Move disk 1 from", source, "to", destination)
        return

    tower_of_hanoi(n - 1, source, destination, auxiliary)

    print("Move disk", n, "from", source, "to", destination)

    tower_of_hanoi(n - 1, auxiliary, source, destination)


n = 3

print("Sequence of moves:")
tower_of_hanoi(n, "A", "B", "C")
```

## Execution Screenshot

### EXECUTION SCREENSHOT



# EXPERIMENT 9

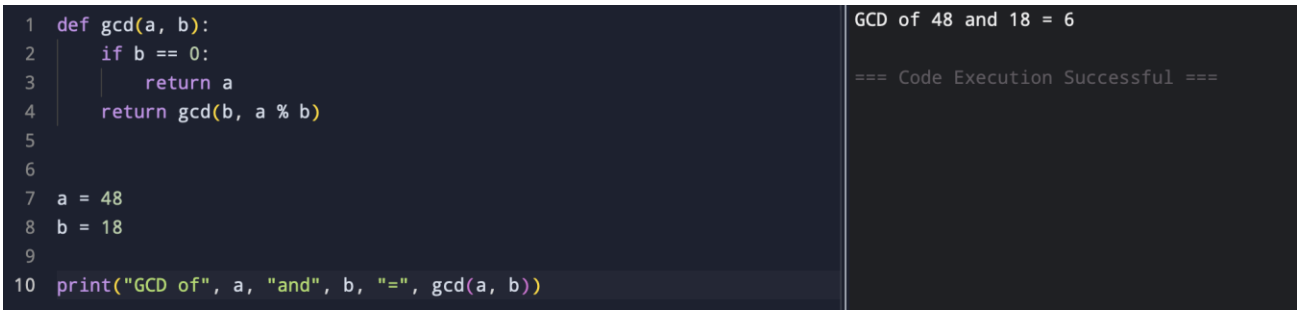
# GCD (EUCLIDEAN ALGORITHM)

## Python Program

```
def gcd(a, b):
    if b == 0:
        return a
    return gcd(b, a % b)


a = 48
b = 18

print("GCD of", a, "and", b, "=", gcd(a, b))
```

## Execution Screenshot

### EXECUTION SCREENSHOT



# EXPERIMENT 10

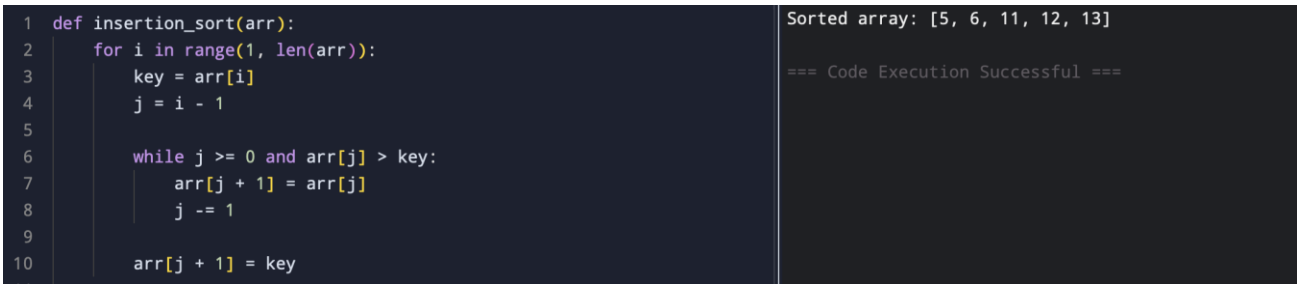
# INSERTION SORT

## Python Program

```
def insertion_sort(arr):
    for i in range(1, len(arr)):
        key = arr[i]
        j = i - 1

        while j >= 0 and arr[j] > key:
            arr[j + 1] = arr[j]
            j -= 1

        arr[j + 1] = key

    return arr


arr = [12, 11, 13, 5, 6]

print("Sorted array:", insertion_sort(arr))
```

## Execution Screenshot

### EXECUTION SCREENSHOT



# EXPERIMENT 11

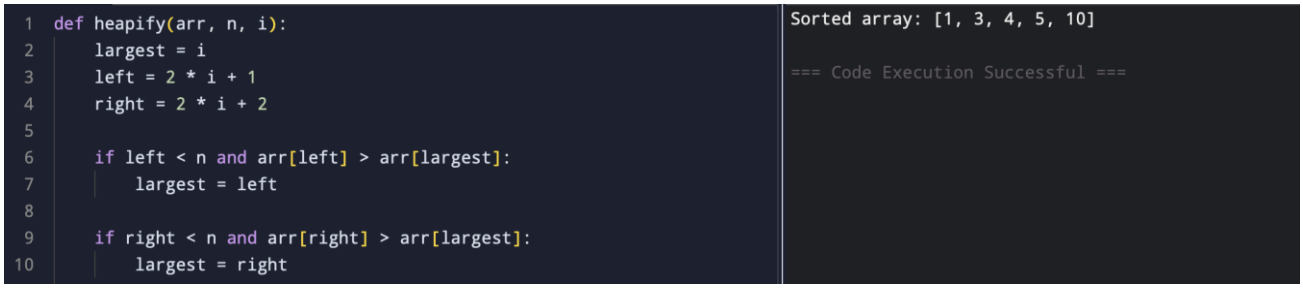
# HEAP SORT

## Python Program

```
def heapify(arr, n, i):
    largest = i
    left = 2 * i + 1
    right = 2 * i + 2

    if left < n and arr[left] > arr[largest]:
        largest = left

    if right < n and arr[right] > arr[largest]:
        largest = right

    if largest != i:
        arr[i], arr[largest] = arr[largest], arr[i]
        heapify(arr, n, largest)


def heap_sort(arr):
    n = len(arr)

    for i in range(n // 2 - 1, -1, -1):
        heapify(arr, n, i)

    for i in range(n - 1, 0, -1):
        arr[0], arr[i] = arr[i], arr[0]
        heapify(arr, i, 0)

    return arr


arr = [4, 10, 3, 5, 1]

print("Sorted array:", heap_sort(arr))
```

## Execution Screenshot

### EXECUTION SCREENSHOT



# EXPERIMENT 12

# COUNTING INVERSIONS

## Python Program

```
def merge_and_count(left, right):
    result = []
    i = j = inv_count = 0

    while i < len(left) and j < len(right):
        if left[i] <= right[j]:
            result.append(left[i])
            i += 1
        else:
            result.append(right[j])
            inv_count += len(left) - i
            j += 1

    result.extend(left[i:])
    result.extend(right[j:])

    return result, inv_count


def count_inversions(arr):
    if len(arr) <= 1:
        return arr, 0

    mid = len(arr) // 2
    left, left_count = count_inversions(arr[:mid])
    right, right_count = count_inversions(arr[mid:])
    merged, split_count = merge_and_count(left, right)

    return merged, left_count + right_count + split_count


arr = [2, 4, 1, 3, 5]

_, count = count_inversions(arr)

print("Number of inversions:", count)
```

## Execution Screenshot

### EXECUTION SCREENSHOT

```
def merge_and_count(left, right):
    result = []
    i = j = inv_count = 0

    while i < len(left) and j < len(right):
        if left[i] <= right[j]:
            result.append(left[i])
            i += 1
```

```
Number of inversions: 3

=== Code Execution Successful ===
```

# EXPERIMENT 13

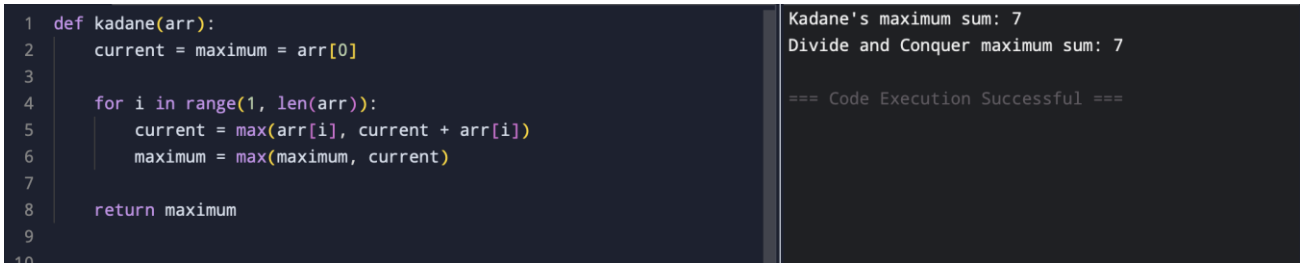
# MAXIMUM SUBARRAY SUM

## Python Program

```
def kadane(arr):
    current = maximum = arr[0]

    for i in range(1, len(arr)):
        current = max(arr[i], current + arr[i])
        maximum = max(maximum, current)

    return maximum


def divide_and_conquer(arr, low, high):
    if low == high:
        return arr[low]

    mid = (low + high) // 2

    left_sum = divide_and_conquer(arr, low, mid)
    right_sum = divide_and_conquer(arr, mid + 1, high)

    left_max = float("-inf")
    total = 0
    for i in range(mid, low - 1, -1):
        total += arr[i]
        left_max = max(left_max, total)

    right_max = float("-inf")
    total = 0
    for i in range(mid + 1, high + 1):
        total += arr[i]
        right_max = max(right_max, total)

    return max(left_sum, right_sum, left_max + right_max)


arr = [-2, -3, 4, -1, -2, 1, 5, -3]

print("Kadane's maximum sum:", kadane(arr))
print("Divide and Conquer maximum sum:",
      divide_and_conquer(arr, 0, len(arr) - 1))
```

## Execution Screenshot

### EXECUTION SCREENSHOT



```python
def kadane(arr):
    current = maximum = arr[0]

    for i in range(1, len(arr)):
        current = max(arr[i], current + arr[i])
        maximum = max(maximum, current)

    return maximum
```

```
Kadane's maximum sum: 7
Divide and Conquer maximum sum: 7

=== Code Execution Successful ===
```

# EXPERIMENT 14

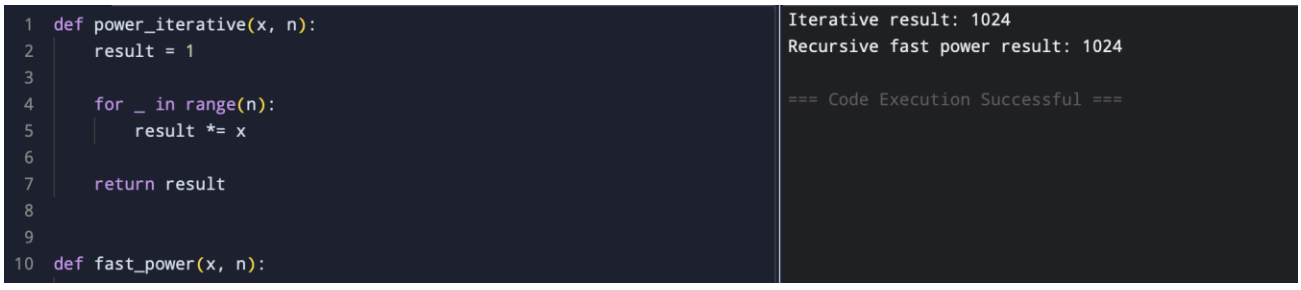
# EXPONENTIATION

## Python Program

```
def power_iterative(x, n):
    result = 1

    for _ in range(n):
        result *= x

    return result


def fast_power(x, n):
    if n == 0:
        return 1

    half = fast_power(x, n // 2)

    if n % 2 == 0:
        return half * half
    else:
        return x * half * half


x = 2
n = 10

print("Iterative result:", power_iterative(x, n))
print("Recursive fast power result:", fast_power(x, n))
```

## Execution Screenshot

### EXECUTION SCREENSHOT



# EXPERIMENT 15

# CLOSEST PAIR OF POINTS

## Python Program

```
import math


def distance(p1, p2):
    return math.sqrt((p1[0] - p2[0]) ** 2 +
                     (p1[1] - p2[1]) ** 2)


def closest_pair(points):
    min_dist = float("inf")
    pair = None

    for i in range(len(points)):
        for j in range(i + 1, len(points)):
            d = distance(points[i], points[j])

            if d < min_dist:
                min_dist = d
                pair = (points[i], points[j])

    return pair, min_dist


points = [(1, 2), (4, 5), (7, 8), (3, 1)]

pair, min_dist = closest_pair(points)

print("Closest pair:", pair)
print("Distance:", min_dist)
```

## Execution Screenshot

### EXECUTION SCREENSHOT